arXiv:2605.21330v1 [cs.RO] 20 May 2026

Learning Robust Dexterous In-Hand Manipulation from Joint Sensors with Proprioceptive Transformer

Senlan Yao[1], Chenyu Yang[1], Jaehoon Kim[1], Aristotelis Sympetheros[1], Robert K. Katzschmann[1]
[1]Soft Robotics Laboratory, ETH Zürich, Switzerland

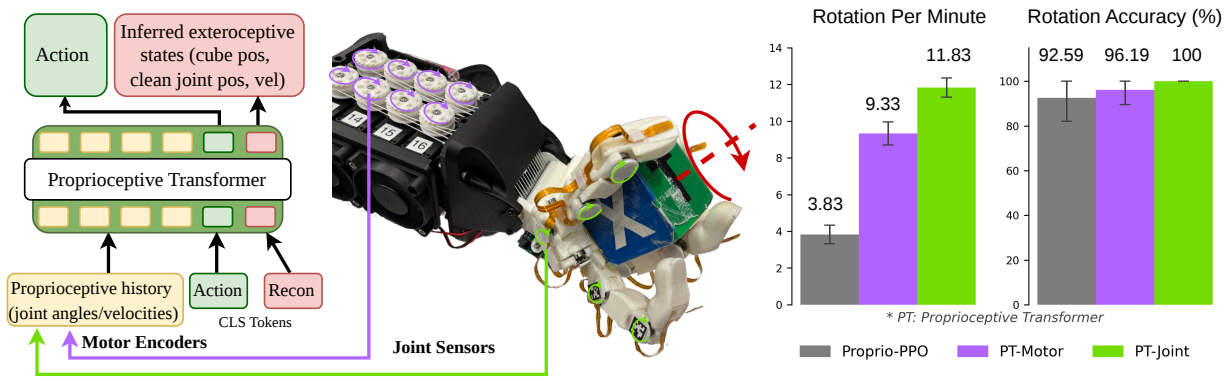


Fig. 1: *Left:* The Proprioceptive Transformer encodes joint position/velocity history and actions, outputting actions and inferred object state. *Middle:* The ORCA hand with 16 AS5600 magnetic angle sensors. *Right:* Our approach achieves 3.1× higher rotation speed and 100% rotation accuracy. Joint sensors improve rotation speed by 26.8% over motor encoders.

***Abstract*—In-hand object manipulation is a fundamental yet challenging capability for dexterous robots. Despite significant progress in dexterous manipulation, existing approaches rely heavily on vision or tactile sensing to track object states, while joint sensing—the most readily available modality on any robotic hand—remains largely overlooked, particularly for tendon-driven hands. In this paper, we study how far joint sensing alone can go by asking: (i) whether motor encoders or direct joint sensing provides better proprioceptive feedback, (ii) how to extract environment information from joint measurements, and (iii) whether joint-only control can achieve competitive real-world performance without external perception. We present the Proprioceptive Transformer (PT), an exteroceptive-free approach for continuous cube rotation on a tendon-driven dexterous hand that uses only joint sensing feedback. A teacher policy is first trained via reinforcement learning with privileged object information, then distilled into PT, which operates solely on joint position and velocity histories. The Transformer architecture effectively extracts implicit object state information from temporal patterns in joint sensor readings. Experiments on the real ORCA hand show that our approach achieves 3.1× higher rotation speed than baselines. We also demonstrate that our PT achieves a 23.4% lower RMSE for cube position estimation than the MLP baseline, indicating superior extraction of exteroceptive information from proprioceptive sources.**

I. Introduction

Dexterous manipulation with robotic hands represents one of the most challenging frontiers in robotics research [1]. The ability to manipulate objects with human-like dexterity has long been a goal of robotics, yet achieving this capability remains elusive due to high-dimensional action spaces, complex contact dynamics, and the need for precise sensory feedback. Among various manipulation tasks, in-hand object reorientation—the ability to rotate and reposition an object within a robotic hand—has emerged as a canonical benchmark for evaluating dexterous manipulation capabilities [2].

Recent advances in deep reinforcement learning (RL) have shown strong performance in learning complex manipulation policies. OpenAI et al. [3] demonstrated that RL agents could learn to reorient a cube to arbitrary target orientations. This achievement relied heavily on large-scale parallel simulation, domain randomization for sim-to-real transfer [4], and sophisticated vision-based perception systems to track the cube's pose throughout the manipulation process.

However, the reliance on external vision systems introduces several practical limitations. Vision-based approaches require careful camera placement, are susceptible to occlusion during manipulation, and demand substantial computational resources for real-time pose estimation [5]. Furthermore, the sim-to-real gap for visual perception can be significant, as rendering realistic images in simulation that match real-world conditions remains challenging [6]. These factors motivate the exploration of alternative sensing modalities that could provide robust and efficient feedback for dexterous manipulation tasks.

Proprioceptive sensing—the use of internal sensors to perceive the robot's own state—is the most basic and widely available sensing modality in robotic hands, yet it is still under-explored for dexterous in-hand manipulation [7]. Joint positions and velocities are always available and exhibit a small sim-to-real gap. This motivates three questions. First, how much exteroceptive information can be inferred solely

from temporal joint measurements, and which policy architecture best captures this information? Second, for tendon-driven hands, is it better to use motor encoder signals or direct joint sensing as proprioceptive feedback, given transmission effects such as cable stretch, friction, and backlash? Third, can a controller trained from joint feedback alone achieve strong real-world in-hand manipulation performance without external vision or tactile sensing?

To answer these questions, we develop the Proprioceptive Transformer (PT) for continuous cube rotation on the ORCA tendon-driven hand. PT is trained through a teacher-student pipeline: a teacher policy learns with privileged object state in simulation, and a student transformer is distilled to operate only on joint position and velocity histories. Using direct finger-joint magnetic sensors [8], our method avoids motor-side transmission mismatch and learns to extract implicit object state from proprioceptive sequences. Our key contributions are:

- A Transformer-based policy that extracts implicit object state from joint sensor history via self-attention, trained with an auxiliary reconstruction objective through teacher-student distillation. We show that this architecture outperforms MLP and LSTM alternatives, and that the reconstruction loss is critical for robust manipulation.
- Experimental evidence that direct joint sensing via magnetic angle sensors provides superior proprioceptive feedback compared to motor encoder readings, effectively bridging the sim-to-real gap caused by tendon transmission nonlinearities.
- Real-hardware validation on the ORCA hand demonstrating that joint-only control achieves $3.1\times$ higher rotation speed (11.83 vs. 3.83 RPM) with 100% accuracy and zero drops, and that joint sensors can implicitly detect object presence and size through position tracking errors.

II. Related Work

A. Dexterous In-Hand Manipulation

Dexterous in-hand manipulation has been a longstanding challenge in robotics. Traditional approaches relied on analytical models and trajectory optimization, requiring precise knowledge of object geometry, contact mechanics, and hand kinematics [9], [10]. However, these model-based methods struggled with the inherent uncertainty of contact dynamics and required extensive manual engineering for each new task.

The advent of deep reinforcement learning (RL) has transformed the field. OpenAI et al. [3] demonstrated that RL agents could learn to reorient a cube to arbitrary target orientations using the Shadow Dexterous Hand. This work leveraged massive parallel simulation and domain randomization to train policies that transferred to real hardware. Building on this success, Akkaya et al. [11] showed that similar techniques could solve the Rubik's cube with a robotic hand. Subsequent works have pushed these boundaries further: DeXtreme [5] achieved more agile in-hand manipulation with improved sim-to-real transfer, while Bi-DexHands [2] extended these capabilities to bimanual dexterous manipulation.

B. From Vision-Based to Proprioceptive Approaches

Early successes in cube reorientation relied heavily on vision-based perception systems. OpenAI et al. [3] employed a sophisticated multi-camera setup with pose estimation pipelines to track the cube's position and orientation throughout manipulation. While effective, this approach introduced several practical limitations: vulnerability to occlusion during finger movements, sensitivity to lighting conditions, computational overhead for real-time pose estimation, and a significant sim-to-real gap in visual appearance [4], [6].

These challenges have motivated a shift toward proprioceptive sensing—using the robot's internal joint sensors rather than external cameras. Proprioceptive observations offer several advantages: they are inherently available in robotic systems, immune to visual occlusions, computationally efficient, and exhibit minimal sim-to-real gap since joint encoder readings are nearly identical in simulation and reality [7]. Kumar et al. [12] demonstrated that complex manipulation behaviors could emerge from proprioceptive feedback combined with tactile sensing. Recent work by Yin et al. [13] showed that in-hand rotation could be achieved using only proprioceptive and tactile feedback, without any visual input.

C. Joint Sensor Feedback in Dexterous Manipulation

A critical distinction exists between using motor commands (or motor encoder readings) versus direct joint sensor measurements as proprioceptive feedback. Motor-side measurements can differ significantly from actual joint positions due to transmission effects including gear backlash, cable stretch, and joint compliance [14], [15]. These discrepancies are particularly problematic for dexterous hands with complex tendon-driven or compliant mechanisms, where the relationship between motor and joint positions is highly nonlinear [16].

Direct joint sensor feedback—measuring positions and velocities at the joint output rather than the motor input—provides more accurate state information. This is especially relevant for hands like the ORCA hand, which features rolling contact joints and tendon-driven actuation [8], [17]. Pitz et al. [18] demonstrated that accurate joint sensing is crucial for learning fine manipulation skills on tendon-driven hands. By using joint encoder readings as policy inputs, we can achieve more precise state estimation and reduce the sim-to-real gap that arises from unmodeled transmission dynamics [19].

D. Teacher-Student Learning with Transformers

Teacher-student learning has emerged as a powerful paradigm for robot control, particularly for sim-to-real transfer. In this framework, a teacher policy is first trained in simulation with access to privileged information (such as ground-truth object states), and then a student policy learns to imitate the teacher using only observations available on the real robot [20], [21]. Kumar et al. [22] introduced Rapid Motor Adaptation (RMA), which uses a teacher-student framework to enable robust locomotion across diverse terrains.

Transformer architectures have shown strong performance in this context. Unlike recurrent networks, transformers can

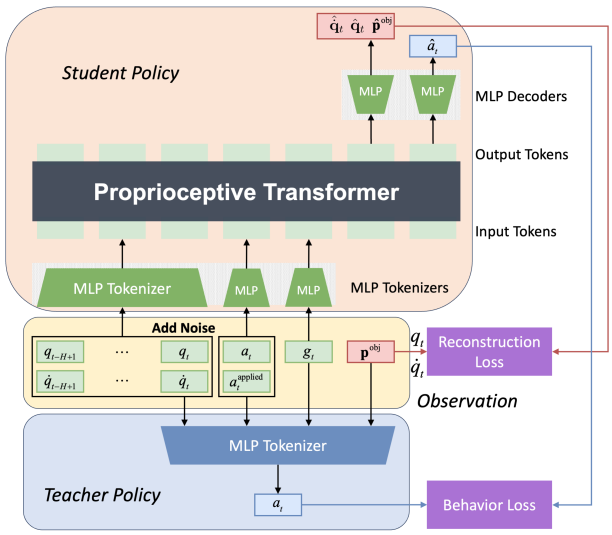


Fig. 2: Teacher-student distillation pipeline. The teacher policy is trained via RL with privileged object state in simulation. The student employs a Proprioceptive Transformer encoder to process joint sensor history via self-attention, and is trained to both imitate teacher actions and reconstruct object states and clean joint states from proprioceptive observations alone.

effectively model long-range temporal dependencies through self-attention mechanisms [23]. For robot control, this capability is crucial: the policy can attend to relevant historical observations to infer hidden states such as object pose and contact dynamics. Chen et al. [24] introduced Decision Transformer, framing RL as sequence modeling, while Janner et al. [25] proposed Trajectory Transformer for offline RL. These approaches have been extended to robotics: Brohan et al. [26] demonstrated RT-1, a transformer-based policy for real-world robot manipulation, and Radosavovic et al. [27] achieved impressive humanoid locomotion using transformer policies.

For cube reorientation specifically, transformer-based policies offer an important advantage: they can learn to implicitly estimate the cube's state from sequences of proprioceptive observations, potentially eliminating the need for explicit pose estimation. These advances suggest that combining teacher-student learning with transformer architectures is an effective direction for proprioceptive-based cube manipulation.

III. Methodology

A. Overview

We propose a three-stage approach for learning cube reorientation using only proprioceptive feedback, as illustrated in Fig. 2. First, a teacher policy is trained with reinforcement learning (PPO) using privileged information including ground-truth object pose. Second, a student policy with a proprioceptive transformer architecture is trained to imitate the teacher using only noisy joint sensor readings. Finally, PT is deployed on the real ORCA hand with zero-shot sim-to-real transfer.

B. Training Environment

We use Isaac Lab as our simulation platform, running 8192 parallel environments of the ORCA [8] dexterous hand. The simulation operates at 120 Hz while the policy runs at 20 Hz (decimation factor of 6), matching the real hardware control frequency. Each episode has a maximum duration of 32 seconds.

Task Definition. The task requires continuous rotation of a 55mm cube following a commanded angular velocity $\boldsymbol{\omega}^* \in \mathbb{R}^3$, constrained to rotation around the z-axis with magnitude up to 1.5 rad/s.

Domain Randomization. To facilitate sim-to-real transfer, we randomize physical properties of both the robot and object. For the robot, we scale friction coefficients (0.8–2.0×), link masses (0.95–1.05×), and randomize joint stiffness/damping parameters using log-uniform distributions. For the object, we randomize friction (0.3–1.0), mass (scaled by 0.5–1.5×), and initial pose with full SO(3) rotation sampling.

C. Teacher Policy Training

The teacher policy is trained using Proximal Policy Optimization (PPO) [28] with access to privileged information. We represent the policy as a Gaussian distribution.

Observations. The teacher receives an 81-dimensional noise-free observation vector consisting of: normalized joint positions $\mathbf{q}_t \in \mathbb{R}^{17}$, joint velocities $\dot{\mathbf{q}}_t \in \mathbb{R}^{17}$, ground-truth object position $\mathbf{p}_t^{\text{obj}} \in \mathbb{R}^3$ and quaternion $\mathbf{r}_t^{\text{obj}} \in \mathbb{R}^4$ (privileged), goal command $\mathbf{c}_t \in \mathbb{R}^6$, previous action $\mathbf{a}_{t-1} \in \mathbb{R}^{17}$, and previous joint position command $\mathbf{q}_{t-1}^{\text{cmd}} \in \mathbb{R}^{17}$.

Actions. The policy outputs 17-dimensional joint position targets, mapped from $[-1, 1]$ to actuator limits. To ensure smooth motions, we apply exponential moving average filtering:

$$a_t^{\text{applied}} = \alpha \cdot a_t + (1 - \alpha) \cdot a_{t-1}^{\text{applied}} \tag{1}$$

where a_t is the raw policy output at time step t, a_t^{applied} is the smoothed action sent to the actuators, and α controls the smoothing strength.

Reward Function. Our reward consists of tracking rewards and regularization terms:

$$r_t = r_{\text{pos}} + r_{\text{mag}} + r_{\text{dir}} + r_{\text{smooth}} + r_{\text{vel}} + r_{\text{act}} + r_{\text{rate}} \tag{2}$$

Position tracking uses an exponential kernel:

$$r_{\text{pos}} = w_p \cdot \exp\left(-\frac{\|\mathbf{p}^{\text{obj}} - \mathbf{p}^{\text{obj}*}\|}{\sigma_p}\right) \tag{3}$$

where $\mathbf{p}^{\text{obj}}$ and $\mathbf{p}^{\text{obj}*}$ are the current and target object positions, w_p is the reward weight, and σ_p is the temperature parameter.

Angular velocity tracking is decomposed into magnitude and direction:

$$r_{\text{mag}} = w_m \cdot \exp\left(-\frac{|\|\boldsymbol{\omega}\| - \|\boldsymbol{\omega}^*\||}{\sigma_\omega}\right) \tag{4}$$

$$r_{\text{dir}} = w_d \cdot \cos(\boldsymbol{\omega}, \boldsymbol{\omega}^*) \tag{5}$$

where $\boldsymbol{\omega}$ and $\boldsymbol{\omega}^*$ are the current and target angular velocities, w_m and w_d are reward weights, and σ_ω is the temperature parameter.

Regularization terms penalize angular acceleration (r_{smooth}), joint velocities (r_{vel}), action magnitude (r_{act}), and action rate (r_{rate}). The action rate penalty proved critical for producing smooth motions that transfer to hardware. All reward weights are reported in Section IV.

D. Student Policy Training

Student Policy Observation. The student operates without privileged information, receiving only: (1) noisy joint positions and velocities over a configurable history window of T steps, (2) previous action and joint position command, and (3) goal command. Joint positions are corrupted with two independent Gaussian noise terms: a per-episode bias $b_j \sim \mathcal{N}(0, \sigma_b^2)$ sampled once at episode start, and a per-step noise $n_t \sim \mathcal{N}(0, \sigma_n^2)$ sampled at each time step. Joint velocities are corrupted with per-step Gaussian noise.

Network Architecture. As shown in Fig. 2, each observation group is first projected to a 256-dimensional token embedding via a linear layer, producing 10 tokens from the proprioceptive history and 1 token each from the action context and command groups (12 tokens total). Sinusoidal positional embeddings are added across the full sequence. A Transformer encoder (3 layers, 4 heads, feedforward dim 1024) processes the sequence with learnable `[CLS]` query tokens appended, which are mapped through separate linear heads to predict actions $\hat{\mathbf{a}}_t \in \mathbb{R}^{17}$ and auxiliary reconstructions of object position and clean joint states.

Training Losses. The behavior cloning loss minimizes the L2 distance between student and teacher actions:

$$\mathcal{L}_{\text{BC}} = \|\hat{\mathbf{a}}_t - \mathbf{a}_t^{\text{teacher}}\|_2^2 \tag{6}$$

where $\hat{\mathbf{a}}_t$ is the student's predicted action and $\mathbf{a}_t^{\text{teacher}}$ is the teacher's action at time step t.

The auxiliary reconstruction loss encourages the Transformer to encode implicit object state and denoise joint measurements from proprioceptive history:

$$\mathcal{L}_{\text{recon}} = \lambda_p \|\hat{\mathbf{p}}_t^{\text{obj}} - \mathbf{p}_t^{\text{obj}}\|_2^2 + \lambda_q \|\hat{\mathbf{q}}_t - \mathbf{q}_t\|_2^2 + \lambda_v \|\hat{\dot{\mathbf{q}}}_t - \dot{\mathbf{q}}_t\|_2^2 \tag{7}$$

where $\hat{\mathbf{p}}_t^{\text{obj}}$ is the predicted object position, $\hat{\mathbf{q}}_t$ and $\hat{\dot{\mathbf{q}}}_t$ are the predicted clean joint positions and velocities, and unhatted quantities denote ground-truth values available only in simulation. λ_p, λ_q, λ_v weight each term respectively. The total loss is $\mathcal{L} = \mathcal{L}_{\text{BC}} + \mathcal{L}_{\text{recon}}$.

IV. Experiments

A. Experimental Setup

We evaluate our approach with the ORCA hand [8], a tendon-driven dexterous manipulator with 17 actuated degrees of freedom (16 finger joints and 1 wrist joint) powered by Dynamixel servo motors, as shown in Fig. 3. The hand is mounted in a palm-up configuration. For proprioceptive sensing, 16 magnetic angle sensors (AS5600, ams OSRAM AG) are integrated at the finger joints, measuring joint angles directly independent of tendon stretch or motor backlash. The 16 AS5600 sensors are mounted at the finger joints, so the wrist joint angle is not directly measured. During deployment, the wrist position is fixed at its zero configuration. Joint velocities are computed by differentiating sensor readings. The control system runs at 20 Hz via ROS2. The trained policy is deployed with direct sim-to-real transfer, taking joint positions and velocities as input and outputting joint position targets to the low-level PD controller. We evaluate on two cube sizes: a 55 mm medium cube and a 65 mm large cube, to test generalization across object dimensions.

Implementation Details. All training is conducted in Isaac Lab with 8192 parallel environments on a single NVIDIA RTX 4070 Ti SUPER GPU.

The teacher policy uses a four-layer MLP with hidden dimensions [512, 512, 256, 128] and ELU activations, trained for 10,000 PPO iterations with learning rate 10^{-4}. The tracking reward weights are position ($w_p = 10$, $\sigma_p = 0.02$), angular velocity magnitude ($w_m = 30$), and direction ($w_d = 60$). Regularization penalties include angular acceleration ($w_{\text{smooth}} = 1$), joint velocity ($w_{\text{vel}} = -10^{-5}$), action magnitude ($w_{\text{act}} = -2 \times 10^{-4}$), and action rate ($w_{\text{rate}} = -0.075$).

The student policy (deployed as PT) is trained for 1200 epochs (6000 gradient update steps) with batch size 256, learning rate 10^{-5}, weight decay 0.01, and reconstruction loss weights $\lambda_p = 0.5$, $\lambda_q = 0.3$, $\lambda_v = 0.2$.

Evaluation Metrics. We introduce four metrics: (1) *Rotations Per Minute (RPM)*: complete rotations around the target axis per minute; (2) *Rotation Accuracy (RA)*: ratio of correct-direction rotations to total z-axis rotations; (3) *Drop-Free Success Rate (DFSR)*: ratio of rotations completed without dropping; (4) *Drop Count (DC)*: total drops per trial. Each trial lasts 60 seconds and results are averaged over 3 trials.

Baselines. We denote our method as **Proprioceptive Transformer (PT)**, with two variants: *PT-Joint*, which uses direct joint sensor feedback via AS5600 magnetic angle sensors, and *PT-Motor*, which relies on motor encoder readings only. We compare against: (1) *Proprio-PPO*: PPO policy using only joint positions and velocities without object information; (2) *Extero-PPO*: PPO policy trained with ground-truth object pose in simulation. During real-world deployment, the object pose is detected from camera images similar to [5].

B. Proprioceptive Transformer vs. PPO Baselines

Each policy is evaluated over 3 trials of 60 seconds each. Table I presents the quantitative comparison on the real robot. Both PT variants significantly outperform all baselines across both cube sizes.

Compared to Proprio-PPO, PT-Joint achieves 3.1× improvement in RPM (11.83 vs. 3.83 for medium cube) while maintaining perfect rotation accuracy and zero drops. Notably, Proprio-PPO exhibits considerable variance in RA (92.59±10.47%) and DFSR (89.74±14.51%), whereas both PT variants achieve consistent 100% accuracy with zero

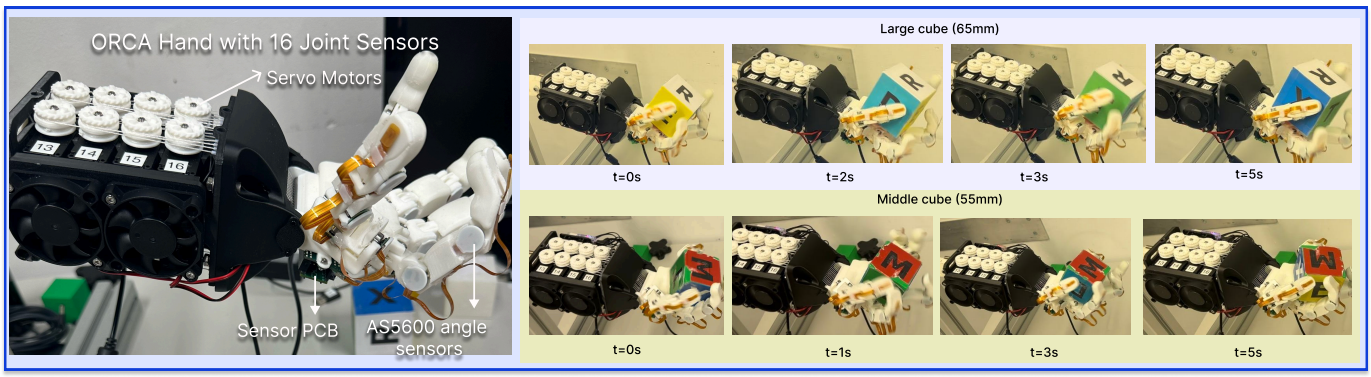


Fig. 3: *Left:* The ORCA hand in our real-world experiment. Magnetic angle sensors are embedded in the joints for direct joint sensing. The antagonistic tendon design allows the joint angles to be estimated from motor angle readings. *Right:* Continuous cube rotation on 55 mm and 65 mm cubes with our PT-Joint Policy.

TABLE I: Performance comparison on cube rotation task. Best results in **bold**.

Medium Cube (55 mm)				
Policy	RPM↑	RA(%)↑	DFSR(%)↑	DC↓
Proprio-PPO	3.83$\pm$0.51	92.59$\pm$10.47	89.74$\pm$14.51	0.33$\pm$0.58
Extero-PPO	3.08$\pm$0.12	**100**$\pm$0	**100**$\pm$0	**0**
Ours				
PT-Motor	9.33$\pm$0.63	96.19$\pm$6.60	98.10$\pm$3.30	0.33$\pm$0.58
PT-Joint	**11.83**$\pm$0.52	**100**$\pm$0	**100**$\pm$0	**0**
Large Cube (65 mm)				
Policy	RPM↑	RA(%)↑	DFSR(%)↑	DC↓
Proprio-PPO	5.17$\pm$0.12	86.98$\pm$9.21	**100**$\pm$0	**0**
Extero-PPO	4.83$\pm$0.12	**100**$\pm$0	**100**$\pm$0	**0**
Ours				
PT-Motor	8.53$\pm$1.48	**100**$\pm$0	96.34$\pm$5.18	0.33$\pm$0.58
PT-Joint	**11.33**$\pm$0.12	**100**$\pm$0	**100**$\pm$0	**0**

variance, indicating superior robustness to observation noise and domain shift. These advantages generalize to the large cube, where PT-Joint achieves 11.33 RPM—2.2× and 2.3× higher than Proprio-PPO and Extero-PPO, respectively—demonstrating robust transfer across object sizes without retraining.

PT-Joint also surpasses Extero-PPO despite the latter having access to ground-truth object poses—achieving 3.8× higher RPM (11.83 vs. 3.08) for the medium cube. This suggests that implicit state estimation from proprioceptive history can be more effective than explicit but noisy pose feedback in sim-to-real transfer.

Comparing PT-Joint and PT-Motor, direct joint sensing provides a consistent 26.8% RPM improvement (11.83 vs. 9.33 for medium cube), confirming that joint sensors better capture the true kinematic state by bypassing tendon transmission nonlinearities. Table II further examines this comparison across different observation window sizes, showing that PT-Joint maintains 100% RA and DFSR at all window sizes while PT-Motor degrades significantly at smaller windows.

C. Implicit Object Detection via Joint Sensors

To better understand how the Proprioceptive Transformer extracts exteroceptive information from proprioceptive observations—and whether direct joint sensing provides a more informative signal than motor encoders—we analyze how joint sensors implicitly encode object information by examining the relationship between joint commands and actual joint positions under different object conditions. Fig. 4 shows scatter plots for representative joints across four conditions collected in simulation: no cube, 45 mm small cube, 55 mm medium cube, and 65 mm large cube.

We observe two distinct effects as cube size increases:

Volume Effect. Larger cubes physically block fingers from reaching commanded positions. When a finger encounters the cube surface, it cannot complete the commanded motion, resulting in position errors. This is particularly visible in MCP joints, where different object volumes lead to distinct grasp postures, as highlighted by the circled regions in Fig. 4(c) and (d).

Weight Effect. Heavier cubes exert more downward force on the fingers. Under the same PD controller gains, this increased load leads to larger steady-state position errors, particularly in the thumb joints that support the object from below, as shown by the circled region in Fig. 4(a) and (b).

These effects create characteristic signatures in joint position data that correlate with object size and presence. Without any cube, actual joint positions closely follow commands (data points near $y{=}x$ line). As cube size increases, deviations become progressively larger, forming distinguishable patterns across object conditions, suggesting that joint sensors can implicitly encode object presence and size through position tracking errors—without vision or tactile feedback.

D. Implicit State Estimation via Transformer

To assess whether temporal modeling is critical for reconstruction quality, we compare three encoder architectures: a Transformer (3-layer, 4-head), an MLP (equivalent parameter count), and an LSTM. All variants share the same reconstruction head and training procedure, differing only in how the observation history is processed. We evaluate reconstruction on both object position and joint state (position and velocity),

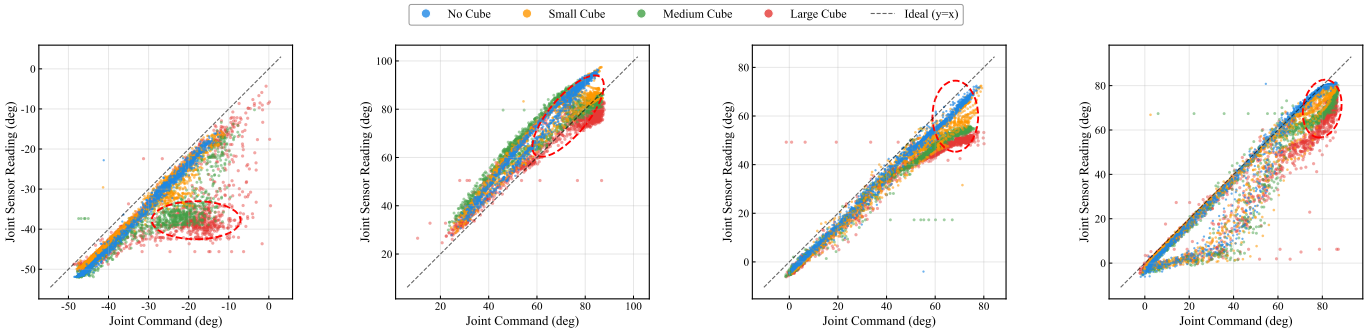
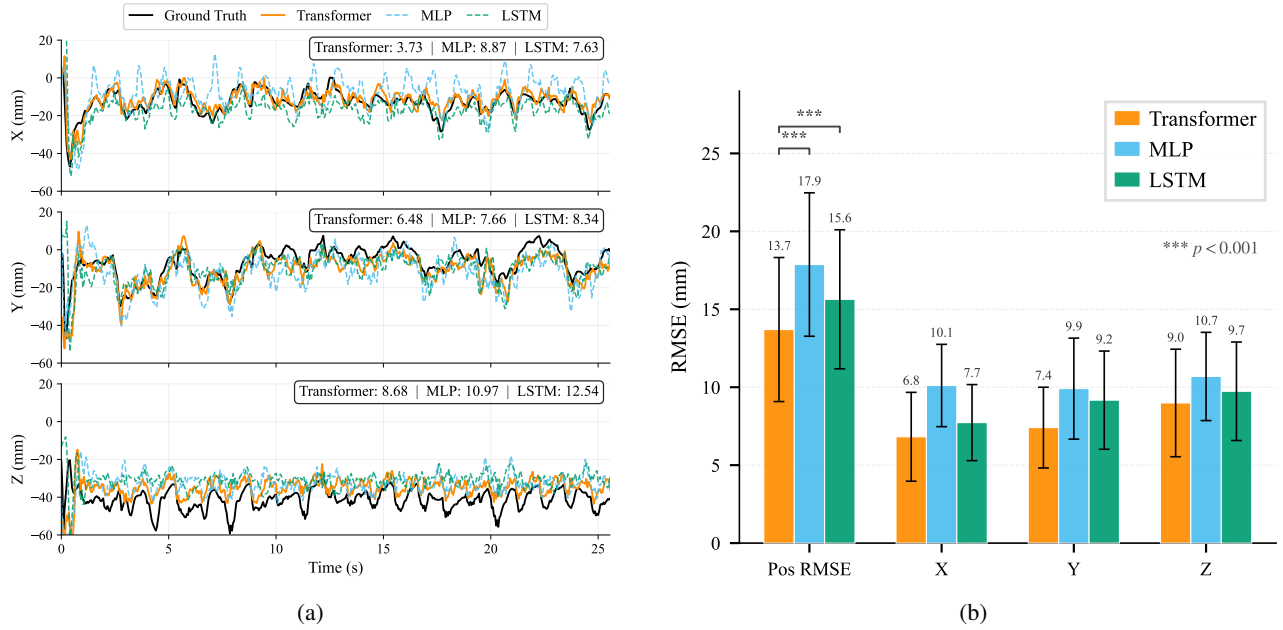


Fig. 4: Joint command vs. actual position for different cube sizes. Deviation from the ideal line ($y{=}x$) increases with cube size due to volume blocking and weight-induced deformation. (a) Thumb MCP. (b) Thumb PIP. (c) Index MCP. (d) Ring MCP.



(a) (b)

Fig. 5: Object position reconstruction from proprioceptive observations alone. (a) Per-axis tracking for a representative environment. (b) Reconstruction RMSE across 32 environments (mean $\pm$ std); the Transformer achieves 13.7 mm, outperforming MLP (17.9 mm, -23.4%) and LSTM (15.6 mm, -12.2%). *** denotes $p < 0.001$ (paired t-test with Bonferroni correction).

as accurate joint state recovery from noisy sensor readings is a prerequisite for reliable sim-to-real transfer.

Object State Reconstruction. Fig. 5a shows a representative reconstruction trajectory for environment 21, where the cube undergoes complex rotation dynamics. The Transformer (orange) closely follows the ground-truth position (black) across all three axes. Notably, during rapid position transitions—such as the sharp drop along Y near $t = 1$ s caused by the cube shifting under finger contact, and the high-frequency oscillations along X between $t = 5$–10 s corresponding to active reorientation phases—the Transformer captures these transient dynamics with minimal delay. In contrast, the MLP (blue) exhibits systematic offset, particularly along the Y-axis where it fails to track the cube's lateral displacement during rotation. The LSTM (green) shows improved temporal sensitivity over MLP but still diverges during rapid state changes, especially along Z where gravity-coupled dynamics require full-sequence attention to model accurately.

Fig. 5b quantifies the reconstruction accuracy across all 32 simulation environments. The Transformer achieves the lowest position RMSE of 13.70 ± 4.62 mm, compared to 17.87 ± 4.60 mm for MLP and 15.64 ± 4.46 mm for LSTM. The Transformer consistently outperforms both baselines across all individual axes, with the largest improvement observed along the X-axis (6.82 vs. 10.11 mm for MLP, a 33% reduction). Pairwise paired t-tests with Bonferroni correction confirm that all differences are statistically significant: Transformer vs. MLP ($p < 0.001$, Cohen's $d = 1.35$) and Transformer vs. LSTM ($p < 0.001$, $d = 1.12$), both with large effect sizes. Notably, the Z-axis exhibits the largest error across all architectures due to gravity-induced finger deflection, yet even this worst case remains well below the cube dimension

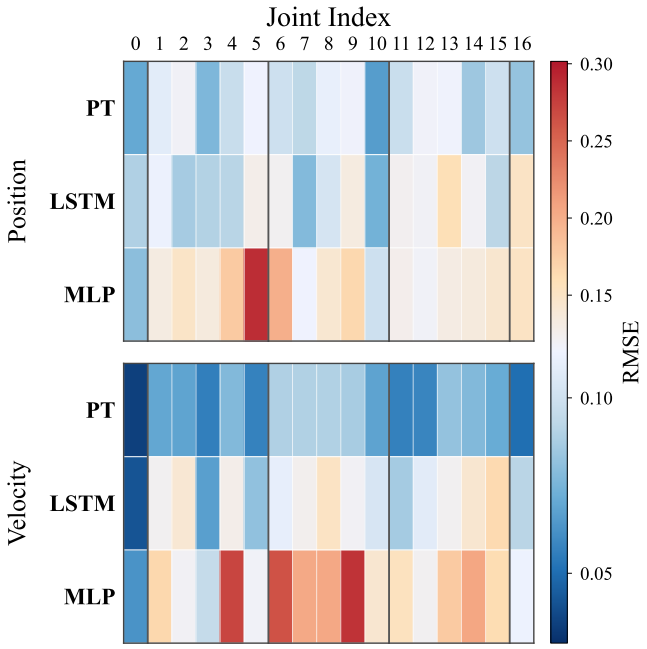


Fig. 6: Per-joint reconstruction RMSE across architectures. Joints are grouped by type: 0 = Wrist; 1–5 = Abduction (Index, Middle, Pinky, Ring, Thumb); 6–10 = Metacarpophalangeal; 11–15 = Proximal interphalangeal; 16 = Distal interphalangeal (Thumb). Each group follows the same finger order.

(55 mm).

Joint State Reconstruction. Beyond object state, we evaluate each architecture's ability to reconstruct noise-free joint positions and velocities from noisy proprioceptive observations. Fig. 6 presents the per-joint RMSE grouped by joint type. The Transformer achieves the lowest mean position RMSE (0.098 rad) and velocity RMSE (0.070 rad/s), outperforming LSTM (0.110 rad, 0.112 rad/s) and MLP (0.146 rad, 0.168 rad/s). MCP joints exhibit the highest error across all architectures, reflecting complex contact dynamics at these joints. The MLP fails most on abduction joints (e.g., Thumb ABD: 0.287 vs. 0.114 rad for Transformer), confirming that temporal context is necessary to disentangle contact-induced deflections from voluntary motion. These results show the Transformer can effectively denoise joint measurements, which is critical for robust sim-to-real transfer.

E. Ablation Study

All ablation experiments are conducted on the real robot using the 55 mm medium cube. We conduct ablation studies on observation window size, sensor feedback configuration, encoder architecture, and reconstruction objective. Tables II and III summarize the results. Each configuration is evaluated over 3 trials of 60 seconds each.

Effect of Sensor Feedback. Across all window sizes, configurations with joint sensors achieve perfect consistency: 100% RA and 100% DFSR. Without sensors, RA drops to 84.75% and DFSR to 94.08% on average. The PT-Joint policy also achieves 6.7% higher RPM (9.27 vs. 8.69) than PT-Motor policy. Direct joint measurements from joint sensors provide reliable feedback that effectively bridges the sim-to-real gap.

TABLE II: Effect of observation window size and sensor feedback.

(a) w/ Sensor (PT-Joint)

Win.	RPM	RA(%)	DFSR(%)	DC
1	$8.33_{\pm 0.14}$	$100_{\pm 0}$	$100_{\pm 0}$	$0_{\pm 0}$
3	$8.17_{\pm 1.04}$	$100_{\pm 0}$	$100_{\pm 0}$	$0_{\pm 0}$
6	$8.75_{\pm 0.25}$	$100_{\pm 0}$	$100_{\pm 0}$	$0_{\pm 0}$
10	$\mathbf{11.83}_{\pm 0.52}$	$100_{\pm 0}$	$100_{\pm 0}$	$0_{\pm 0}$
Mean	*9.27*	*100*	*100*	*0*

(b) w/o Sensor (PT-Motor)

Win.	RPM	RA(%)	DFSR(%)	DC
1	$7.83_{\pm 0.14}$	$97.0_{\pm 5.2}$	$100_{\pm 0}$	$0_{\pm 0}$
3	$9.25_{\pm 1.09}$	$73.9_{\pm 14.3}$	$78.2_{\pm 10.3}$	$2_{\pm 1}$
6	$8.33_{\pm 0.14}$	$71.9_{\pm 7.2}$	$100_{\pm 0}$	$0_{\pm 0}$
10	$9.33_{\pm 0.63}$	$96.19_{\pm 6.60}$	$98.10_{\pm 3.30}$	$0.33_{\pm 0.58}$
Mean	*8.69*	*84.75*	*94.08*	*0.58*

TABLE III: Effect of encoder architecture and reconstruction loss.

Policy	RPM↑	RA(%)↑	DFSR(%)↑	DC↓
No Recon	$4.75_{\pm 0.75}$	$68.9_{\pm 39.1}$	$35.0_{\pm 20.9}$	$5.0_{\pm 1.7}$
MLP	$5.08_{\pm 0.38}$	$68.1_{\pm 19.7}$	$61.5_{\pm 16.8}$	$2.0_{\pm 1.0}$
LSTM	$7.00_{\pm 0.25}$	$95.4_{\pm 8.0}$	$100_{\pm 0}$	$0.0_{\pm 0.0}$
Transformer	$9.33_{\pm 0.63}$	$96.19_{\pm 6.60}$	$98.10_{\pm 3.30}$	$0.33_{\pm 0.58}$

Effect of Window Size. Larger temporal context improves performance. As the window size increases from 1 to 10, the average RPM improves by 31% (8.08 to 10.58, averaged across sensor configurations). A window size of 10 achieves best results across all metrics, demonstrating that the transformer encoder effectively leverages observation history to implicitly estimate object state through self-attention.

Effect of Encoder Architecture and Reconstruction. We evaluate encoder architecture and reconstruction objective under the w/o Sensor configuration (window size 10) to isolate architectural effects from sensor type. The Transformer achieves the highest RPM ($9.33_{\pm 0.63}$) with 96.19% RA and 98.10% DFSR, outperforming LSTM ($7.00_{\pm 0.25}$ RPM) and MLP ($5.08_{\pm 0.38}$ RPM) by 33% and 84%, respectively. This demonstrates that self-attention more effectively extracts implicit object state from proprioceptive history than recurrent or feedforward alternatives, by selectively attending to contact-relevant time steps. Ablating the reconstruction head (No Recon) causes severe degradation—DFSR drops to $35.0_{\pm 20.9}$% with an average of 5 drops per trial—confirming that the auxiliary reconstruction objective is critical for shaping latent representations that capture object dynamics.

V. Conclusion

Our work demonstrates that high-performance in-hand manipulation can be achieved from proprioception alone. We introduced the Proprioceptive Transformer, a teacher-student

framework that distills privileged simulation information into a deployable policy using only joint position and velocity histories, and we showed that direct joint sensing is more effective than motor-side sensing for tendon-driven hands.

On real-world benchmarks with the ORCA hand, PT-Joint achieved a **3.1**× speed gain over the baseline on the medium cube and up to **2.3**× over Extero-PPO on the large cube. Ablation and reconstruction analyses further support these results: removing the reconstruction objective reduced DFSR to **35.0%** with an average of 5 drops, and the PT achieved the lowest object-state reconstruction error 23.4% lower than the MLP baseline.

More broadly, these results support a practical direction for dexterous manipulation: extracting exteroceptive information from onboard proprioception to reduce dependence on external perception stacks. Future work will extend this framework to multi-axis reorientation and more diverse object geometries, and will study how proprioceptive latent-state learning can be combined with sparse exteroceptive cues (such as tactile information) for even broader real-world generalization.

REFERENCES

[1] A. Billard and D. Kragic, "Trends and challenges in robot manipulation," *Science*, vol. 364, no. 6446, 2019.

[2] Y. Chen, T. Wu, S. Wang, X. Feng, J. Jiang, Z. Lu, S. McAleer, H. Dong, S.-C. Zhu, and Y. Yang, "Towards human-level bimanual dexterous manipulation with reinforcement learning," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 5150–5163.

[3] OpenAI, M. Andrychowicz, B. Baker, M. Chociej, R. Józefowicz, B. McGrew, J. Pachocki, A. Petron, M. Plappert, G. Powell, A. Ray, J. Schneider, S. Sidor, J. Tobin, P. Welinder, L. Weng, and W. Zaremba, "Learning dexterous in-hand manipulation," *The International Journal of Robotics Research*, vol. 39, no. 1, pp. 3–20, 2020.

[4] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain randomization for transferring deep neural networks from simulation to the real world," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2017, pp. 23–30.

[5] A. Handa, A. Allshire, V. Makoviychuk, A. Petrenko, R. Singh, J. Liu, D. Makoviichuk, K. Van Wyk, A. Zhurkevich, B. Sundaralingam *et al.*, "Dextreme: Transfer of agile in-hand manipulation from simulation to reality," in *IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2023, pp. 5977–5984.

[6] W. Zhao, J. P. Queralta, and T. Westerlund, "Sim-to-real transfer in deep reinforcement learning for robotics: A survey," in *IEEE Symposium Series on Computational Intelligence (SSCI)*, 2020, pp. 737–744.

[7] F. Muratore, F. Ramos, G. Turk, W. Yu, M. Gienger, and J. Peters, "Robot learning from randomized simulations: A review," *Frontiers in Robotics and AI*, vol. 9, 2022.

[8] C. C. Christoph, M. Eberlein, F. Katsimalis, A. Roberti, A. Sympetheros, M. R. Vogt, D. Liconti, C. Yang, B. G. Cangan, R. J. Hinchet, and R. K. Katzschmann, "Orca: An open-source, reliable, cost-effective, anthropomorphic robotic hand for uninterrupted dexterous task learning," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2025.

[9] A. Bicchi, "Hands for dexterous manipulation and robust grasping: A difficult road toward simplicity," *IEEE Transactions on Robotics and Automation*, vol. 16, no. 6, pp. 652–662, 2000.

[10] A. M. Okamura, N. Smaby, and M. R. Cutkosky, "An overview of dexterous manipulation," in *IEEE International Conference on Robotics and Automation (ICRA)*, vol. 1. IEEE, 2000, pp. 255–262.

[11] I. Akkaya, M. Andrychowicz, M. Chociej, M. Litwin, B. McGrew, A. Petron, A. Paino, M. Plappert, G. Powell, R. Ribas *et al.*, "Solving rubik's cube with a robot hand," *arXiv preprint arXiv:1910.07113*, 2019.

[12] V. Kumar, E. Todorov, and S. Levine, "Optimal control with learned local models: Application to dexterous manipulation," in *IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2016, pp. 378–383.

[13] Z.-H. Yin, B. Huang, Y. Qin, Q. Chen, and X. Wang, "Rotating without seeing: Towards in-hand dexterity through touch," in *Robotics: Science and Systems (RSS)*, 2023.

[14] M. W. Spong, "Modeling and control of elastic joint robots," *Journal of Dynamic Systems, Measurement, and Control*, vol. 109, no. 4, pp. 310–318, 1987.

[15] A. Albu-Schäffer, C. Ott, and G. Hirzinger, "Soft robotics: From torque feedback-controlled lightweight robots to intrinsically compliant systems," *IEEE Robotics & Automation Magazine*, vol. 15, no. 3, pp. 20–30, 2008.

[16] Z. Xu and E. Todorov, "Design of a highly biomimetic anthropomorphic robotic hand towards artificial limb regeneration," pp. 3485–3492, 2016.

[17] Y. Toshimitsu, B. Forrai, B. G. Cangan, U. Steger, M. Knecht, S. Weirich, and R. K. Katzschmann, "Getting the ball rolling: Learning a dexterous policy for a biomimetic tendon-driven hand with rolling contact joints," in *IEEE-RAS International Conference on Humanoid Robots (Humanoids)*. IEEE, 2023, pp. 1–7.

[18] J. Pitz, L. Röstel, L. Sievers, and B. Bäuml, "Dexterous tactile in-hand manipulation using a modular reinforcement learning architecture," in *IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2023, pp. 1852–1858.

[19] J. Hwangbo, J. Lee, A. Dosovitskiy, D. Bellicoso, V. Tsounis, V. Koltun, and M. Hutter, "Learning agile and dynamic motor skills for legged robots," *Science Robotics*, vol. 4, no. 26, p. eaau5872, 2019.

[20] J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter, "Learning quadrupedal locomotion over challenging terrain," *Science Robotics*, vol. 5, no. 47, p. eabc5986, 2020.

[21] D. Chen, B. Zhou, V. Koltun, and P. Krähenbühl, "Learning by cheating," in *Conference on Robot Learning (CoRL)*. PMLR, 2020, pp. 66–75.

[22] A. Kumar, Z. Fu, D. Pathak, and J. Malik, "Rma: Rapid motor adaptation for legged robots," in *Robotics: Science and Systems (RSS)*, 2021.

[23] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[24] L. Chen, K. Lu, A. Rajeswaran, K. Lee, A. Grover, M. Laskin, P. Abbeel, A. Srinivas, and I. Mordatch, "Decision transformer: Reinforcement learning via sequence modeling," *Advances in Neural Information Processing Systems*, vol. 34, pp. 15 084–15 097, 2021.

[25] M. Janner, Q. Li, and S. Levine, "Offline reinforcement learning as one big sequence modeling problem," in *Advances in Neural Information Processing Systems*, vol. 34, 2021, pp. 1273–1286.

[26] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gober, K. Hausman, A. Herzog *et al.*, "Rt-1: Robotics transformer for real-world control at scale," in *Robotics: Science and Systems (RSS)*, 2023.

[27] I. Radosavovic, T. Xiao, B. Zhang, T. Darrell, J. Malik, and K. Sreenath, "Real-world humanoid locomotion with reinforcement learning," *Science Robotics*, vol. 9, no. 89, p. eadi9579, 2024.

[28] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," in *arXiv preprint arXiv:1707.06347*, 2017.